

Scentra Fase 11 - Tutorial de Entrenamiento ML

Tutorial Fase 11: Entrenar Modelos ML En Scentra

Scope: SaaS only. Este tutorial aplica a la infraestructura real detectada en `saas-version/`: FastAPI, PostgreSQL, optional ML profile, MLflow, ML service, feature store, auto-labeling y Admin AI Predictivo.

0. Que existe hoy

Scentra Fase 11 ya tiene:

- Eventos canonicos en `saas_intelligence_events`.
- Auto-labels en `saas_ml_auto_labels`.
- Feature store en `saas_intelligence_feature_values`.
- Feature pipelines en `saas_ml_feature_pipeline_runs`.
- Datasets de entrenamiento en `saas_ml_training_datasets`.
- Evaluaciones en `saas_ml_model_evaluations`.
- Model registry en `saas_intelligence_model_registry`.
- Jobs/artifacts/inferencia/drift en tablas `saas_ml_*`.
- ML service opcional con LightGBM, XGBoost, sklearn fallback, MLflow/BentoML y metricas.

Por defecto el runtime productivo sigue usando `baseline_rules`. Los modelos entrenados deben pasar por shadow y canary antes de produccion.

1. Preparar entorno

Usar staging primero.

```
docker compose -f saas-version\docker-compose.saas.yml --profile ml up -d --build
```

Variables esperadas para pruebas ML:

```
SAAS_ML_ENABLED=true
SAAS_ML_SHADOW_INFERENCE_ENABLED=true
SAAS_ML_AUTO_TRAIN_ENABLED=false
SAAS_ML_SERVICE_URL=http://ml-service:8090
SAAS_MLFLOW_TRACKING_URI=http://mlflow:5000
```

No activar `SAAS_ML_AUTO_TRAIN_ENABLED=true` hasta tener labels revisados.

2. Verificar servicios

```
docker compose -f saas-version\docker-compose.saas.yml ps
docker compose -f saas-version\docker-compose.saas.yml exec -T api python -m app_saas.tools.migrate /app/migrations
```

Validar:

- API: `/saas/v1/health`
- Swagger: `/docs`
- Admin: servicio `admin-frontend`
- ML service: GET `http://localhost:<puerto-ml>/health` si el puerto esta expuesto

- MLflow: servicio mlflow
- Qdrant: servicio qdrant

3. Habilitar feature gates para un tenant piloto

Desde Admin AI Predictivo, habilitar solo el tenant piloto:

- intelligence_demo
- ai_premium
- ml_predictions
- el modelo objetivo, por ejemplo lead_scoring_ml, churn_prediction o smart_remarketing
- predictive_recommendations solo si se quieren guardar recomendaciones

Regla: demo para ver, full para persistir/usar cuotas.

4. Recolectar eventos

Fuentes reales:

- CRM mensajes/conversaciones/clientes.
- Billing subscription changes.
- Campanas, broadcasts, triggers, remarketing.
- Webhooks, outbound queues y errores operativos.
- Feedback de predicciones.

Ejecutar el worker de inteligencia desde Admin Operacion o AI Predictivo:

```
POST /saas/v1/admin/operations/intelligence/process
```

Tambien puede correr desde el worker normal. Confirmar que existan filas en:

- saas_intelligence_events
- saas_intelligence_feature_values

5. Generar auto-labels

Desde Admin AI Predictivo usar el panel de Data Intelligence, o llamar:

```
POST /saas/v1/admin/intelligence/auto-labels/generate
```

Labels esperados:

- Lead scoring: conversion, win, compra, reserva, solicitud de precio con cierre.
- Churn: inactividad prolongada, caida de engagement, escalacion no resuelta.
- Smart remarketing: click, respuesta, compra/reserva posterior, recuperacion.
- Operational anomaly: errores webhook, dead-letter, colas degradadas, fallos outbound.

Revisar distribucion antes de entrenar:

- total de labels
- positivos vs negativos
- confidence promedio

- evidencia usada
- tenant_id correcto

Si hay muy pocos positivos, no entrenar todavia.

6. Recomputar features

Desde Admin:

POST /saas/v1/admin/intelligence/feature-pipelines/recompute

Features por modelo:

- Lead scoring: response_time, message_count, asked_for_price, engagement_score, avg_reply_speed, channel_source, followup_count.
- Churn: inactivity_days, negative_sentiment_ratio, response_drop, ticket_frequency, engagement_decline.
- Remarketing: open_rate, click_rate, best_hour, best_channel, campaign_engagement.
- Operational anomaly: webhook_errors, queue_depth, dead_letters, outbound_failure_rate.

Validar que las filas tengan:

- tenant_id
- subject_type
- subject_id
- feature_set_key
- feature_version
- quality_json

7. Construir dataset

Desde Admin:

POST /saas/v1/admin/intelligence/ml-datasets/build

El ML service une:

- saas_ml_auto_labels
- saas_intelligence_feature_values

Salida esperada:

- dataset CSV/manifest en el directorio configurado
- fila en saas_ml_training_datasets
- conteo de samples
- conteo por clase
- columnas/features utilizadas

Minimos recomendados:

- Lead scoring: 500 a 1,000 labels utiles minimo; ideal 5,000+.
- Churn: 300 a 500 minimo; ideal 2,000+.
- Remarketing: 1,000+ outcomes; ideal por canal/industria.

- Operational anomaly: trafico normal suficiente y algunos incidentes conocidos.

8. Entrenar modelo

Desde Admin:

POST /saas/v1/admin/intelligence/ml-training/autolabel

Recomendacion inicial:

- Lead scoring: LightGBM o XGBoost.
- Churn: LightGBM o sklearn si dataset pequeno.
- Remarketing: XGBoost/LightGBM con features de hora/canal.
- Operational anomaly: baseline heuristico + sklearn/LightGBM cuando haya datos.

No usar GPU. No entrenar LLMs propios.

9. Revisar evaluacion

Revisar:

- sample_count
- precision
- recall
- ROC AUC cuando aplique
- matriz de confusion
- feature importances
- drift baseline
- falsos positivos/falsos negativos
- latencia de inferencia

Si el dataset es pequeno, usar el resultado como shadow bootstrap, no como produccion.

10. Registrar en model registry

El entrenamiento puede registrar artifact en estado shadow. Confirmar en Admin:

- model_key
- prediction_type
- stage: shadow
- status: active solo si esta listo para shadow
- artifact uri/path
- evaluation metadata

No marcar production directo.

11. Shadow inference

Activar:

SAAS_ML_SHADOW_INFERENCE_ENABLED=true

Las predicciones productivas siguen usando baseline. El output ML aparece como metadata ml_inference, sin cambiar decision de negocio.

Validar:

- no aumentan errores en API/worker
- latencia aceptable
- fallback a baseline funciona
- no se crean recomendaciones inseguras desde shadow

12. Canary rollout

Cuando shadow pase revision:

1. Registrar modelo como canary aprobado.
2. Empezar con 5-10% de trafico elegible.
3. Monitorear calidad, drift, latencia, feedback y costos.
4. Mantener rollback a baseline.

No usar canary en todos los tenants a la vez.

13. Produccion

Promover a production solo si:

- labels revisados
- evaluacion offline aceptada
- shadow sin regresiones
- canary estable
- rollback probado
- feature gates y quotas correctos
- lenguaje de producto aprobado como decision support

14. Monitoreo continuo

Revisar semanalmente:

- drift
- precision/feedback
- labels nuevos
- distribucion por tenant/industria
- costos de inferencia
- latencia
- errores por proveedor/ML service
- recomendaciones aceptadas o descartadas

15. Cuando reentrenar

Reentrenar cuando:

- cambian industrias/vertical packs
- crece el volumen de labels
- drift alto sostenido
- bajan metricas de precision/recall
- cambia comportamiento por canal
- cambia el producto o pipeline CRM

16. Checklist antes de vender como ML productivo

- Staging con datos reales anonimizados.
- 1 tenant piloto por industria.
- Validacion humana de labels.
- Reporte de evaluacion.
- Shadow aceptado.
- Canary aceptado.
- Rollback documentado.
- Costos y cuotas definidos.
- Legal/privacy revisado.
- Planes/feature flags configurados.

17. Lo que no se necesita ahora

- GPUs.
- Kafka/NATS obligatorio.
- Entrenamiento distribuido.
- Entrenar LLMs propios.
- Datasets privados externos.

18. Riesgo clave

El modelo aprende de la calidad de eventos, labels y features. Si los datos no reflejan conversion, abandono o campanas reales, el modelo puede verse "operativo" pero no ser comercialmente confiable. Usar shadow/canary para separar infraestructura lista de calidad predictiva real.